



Lecture 5

Packages, Recursion, IntLists

CS61B, Spring 2026 @ UC Berkeley

Josh Hug and Kay Ousterhout

Packages

Lecture 5, CS61B, Spring 2026

Packages

Goals: Building a List

IntList and Linked Data Structures

Before we start today, let's talk briefly about packages.

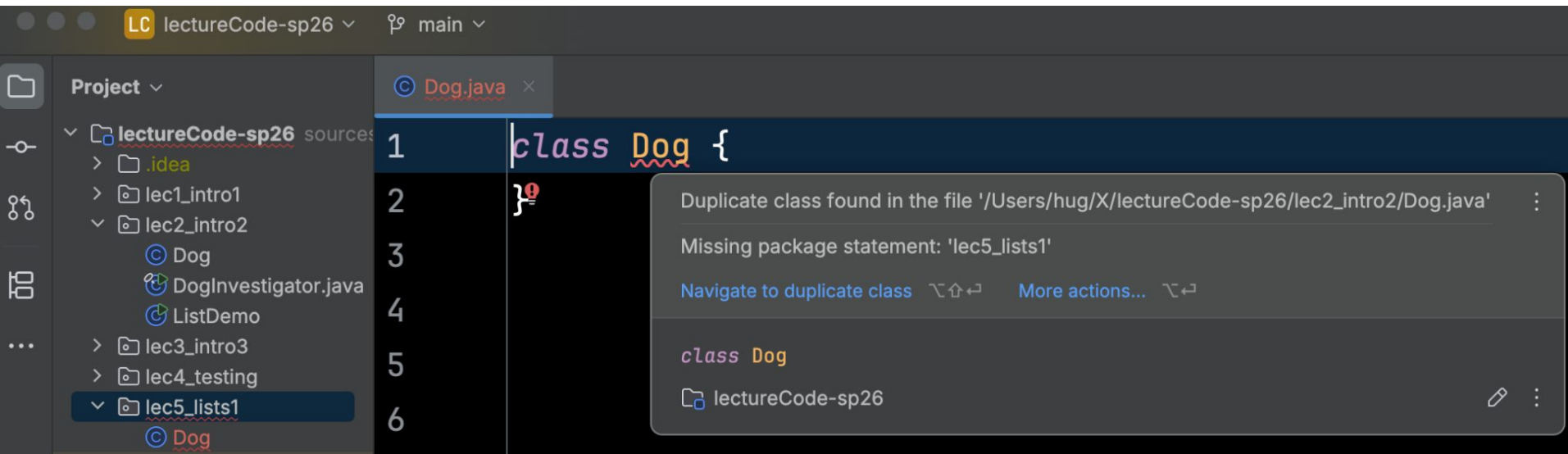
- A bit dull, but I hope this saves you from future trouble.

Duplicate Classes

Suppose I:

- Have Dog.java in the lec2_intro2 folder.
- Have Dog.java in the lec5_lists1 folder.

Java will get mad, because it can see two Dog classes.



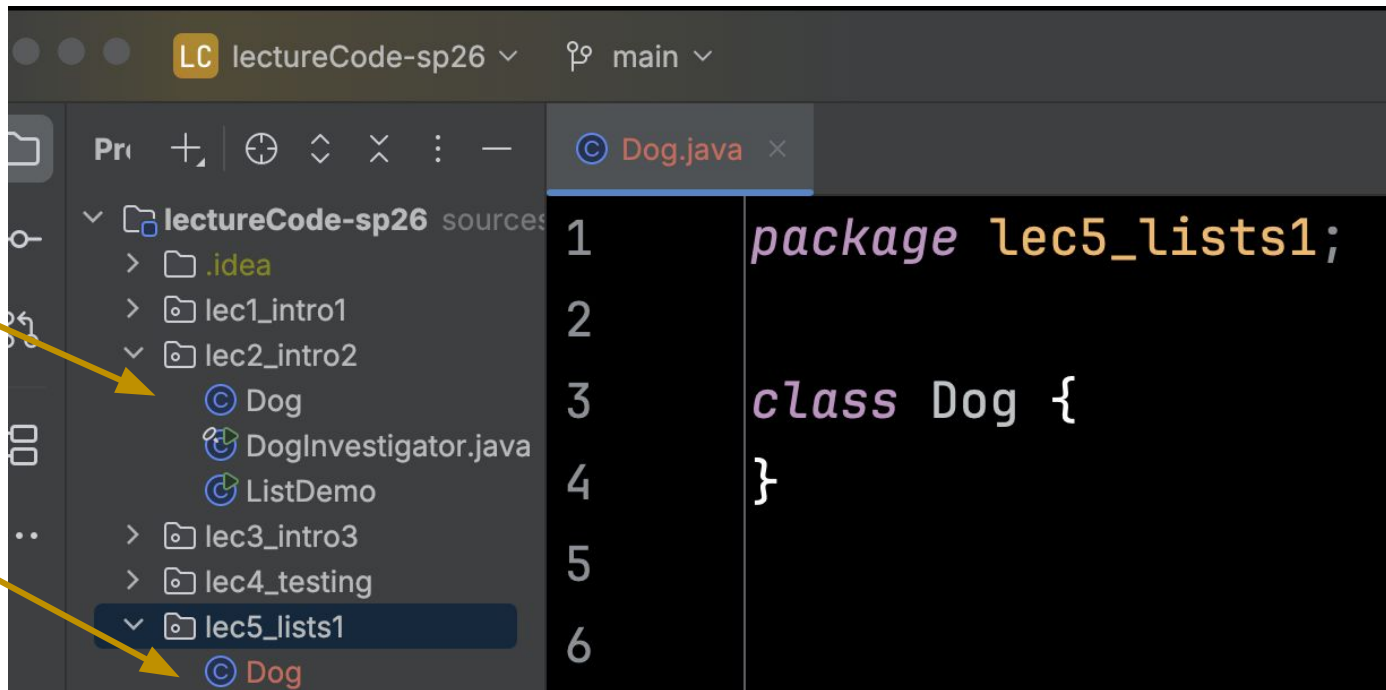
Declaring Code to be Part of a Package

By adding `package lec5_lists1;` to the first line of my file, I give a new “canonical” name to this Dog class.

- The one from `lec2_intro2` is just named `Dog` (because it has no package).
- This one is now named `lec5_lists1.Dog`.

Canonical name is
`Dog`

Canonical name is
`lec5_lists1.Dog`



Importing (is not strictly necessary)

Importing is not required to use code from another package.

- If you don't import, you must give the full canonical name.
- For example, the code below is valid:

```
package lec5_lists1;

public class TestDog {
    @org.junit.jupiter.api.Test
    public void testSort() {
        String[] input = {"string", "apple", "helloworld", "wobbly"};
        String[] expected = {"apple", "helloworld", "string", "wobbly"};

        lec4_testing.Sort.sort(input);
        com.google.common.truth.Truth.assertThat(input).isEqualTo(expected);
    }
}
```

full canonical name

Importing Means “This is Shorthand for”

When you import a class, e.g. `lec4_testing.Sort`, you are saying:

- Whenever I write `Sort`, this is shorthand for `lec4_testing.Sort`.

```
package lec5_lists1;
import lec4_testing.Sort;
import static com.google.common.truth.Truth.assertThat;

public class TestDog {
    @org.junit.jupiter.api.Test
    public void testSort() {
        String[] input = {"string", "apple", "helloworld", "wobbly"};
        String[] expected = {"apple", "helloworld", "string", "wobbly"};

        Sort.sort(input);
        assertThat(input).isEqualTo(expected);
    }
}
```

Importing (is not strictly necessary)

You can also import static members of a class, example below.

- `com.google.common.truth`: Package name.
- `Truth`: A class in the `com.google.common.truth` package:
- `assertThat`: A static method in the `Truth` class.

```
package lec5_lists1;
import lec4_testing.Sort;
import static com.google.common.truth.Truth.assertThat;

public class TestDog {
    @org.junit.jupiter.api.Test
    public void testSort() {
        String[] input = {"string", "apple", "helloworld", "wobbly"};
        String[] expected = {"apple", "helloworld", "string", "wobbly"};

        Sort.sort(input);
        assertThat(input).isEqualTo(expected);
    }
}
```

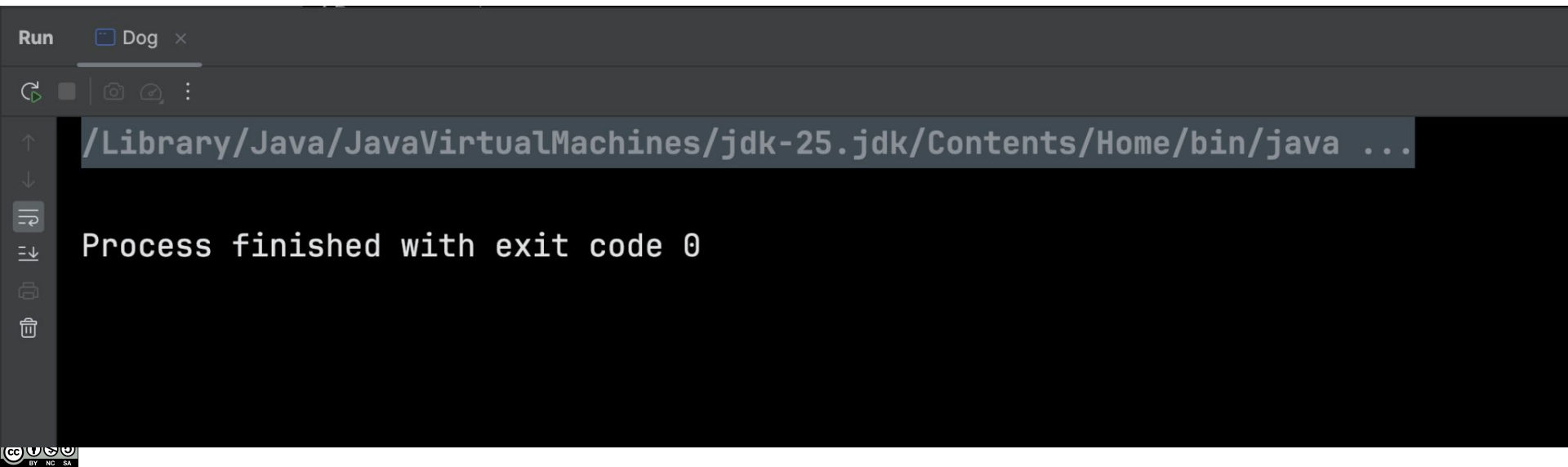

The CLASSPATH

Java won't look at every folder on your computer.

- Too slow, might pick up things you don't want to use.

Instead, it only looks on the CLASSPATH.

- To see your CLASSPATH, click the ... in the terminal after running your code:



The screenshot shows a terminal window with a dark background. At the top, there's a tab labeled 'Run' and a window title 'Dog'. Below the title bar, there's a toolbar with icons for running, debugging, and other actions. The main area of the terminal displays the command `/Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java ...` in a light blue font. Below the command, the text `Process finished with exit code 0` is displayed in a light green font. On the left side of the terminal, there's a vertical toolbar with icons for navigating through the code, including arrows and a magnifying glass.

```
Run Dog x
/Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java ...
Process finished with exit code 0
```

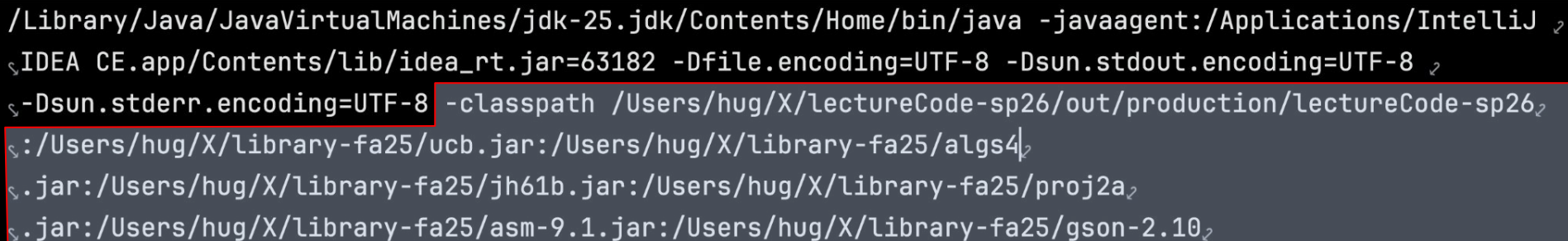
The CLASSPATH

Java won't look at every folder on your computer.

- Too slow, might pick up things you don't want to use.

Instead, it only looks on the CLASSPATH.

- To see your CLASSPATH, click the ... in the terminal after running your code:



```
/Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java -javaagent:/Applications/IntelliJ  
IDEA CE.app/Contents/lib/idea_rt.jar=63182 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8  
-Dsun.stderr.encoding=UTF-8 -classpath /Users/hug/X/lectureCode-sp26/out/production/lectureCode-sp26  
:/Users/hug/X/library-fa25/ucb.jar:/Users/hug/X/library-fa25/algs4  
:/Users/hug/X/library-fa25/jh61b.jar:/Users/hug/X/library-fa25/proj2a  
:/Users/hug/X/library-fa25/asm-9.1.jar:/Users/hug/X/library-fa25/gson-2.10
```

Public vs. Package-Private

In this class, we sometimes declared using `public`, and sometimes without. Difference:

- If you say `public`: Usable by code in other packages.
- If you omit `public`: Usable only by code in the same package.

```
© Dog.java x
1 package lec5_lists1;
2
3 class Dog {
4 }
```

Only usable by other
lec5_lists1 code

```
© Sort.java x
1 package lec4_testing;
2
3 public class Sort {
4     public static void sort(String[] input) {
5         sort(input, 0);
6     }
}
```

Usable by the public, i.e. any
class in any package

Meet the people sitting next to you!

- www.yellkey.com/expert



Goals: Building a List

Lecture 5, CS61B, Spring 2026

Packages

Goals: Building a List

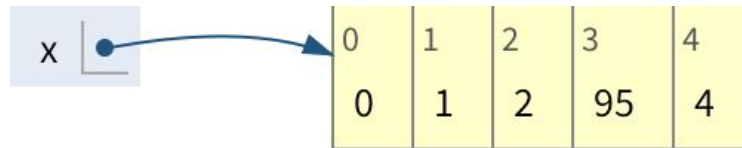
IntList and Linked Data Structures

Declaration and Instantiation of Arrays

Arrays are Objects, and are instantiated using the **new** keyword.

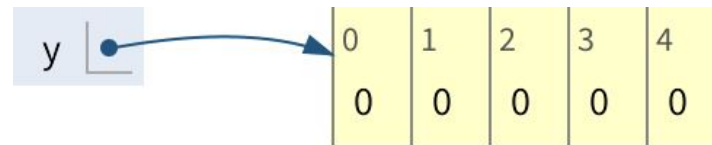
```
int[] x = new int[]{0, 1, 2, 95, 4};
```

- Instantiates an array with specific integers.



```
int[] y = new int[5];
```

- Instantiates an array of empty values.



```
int[] a;
```



- Declaration creates a 64 bit box intended only for storing a reference to an int array. **No object is instantiated.**

What if we want to add and remove items?

Unlike Python, lists are not built directly into the Java language.

```
import java.util.List;
import java.util.LinkedList;
List<String> L = new LinkedList<>();
L.add("a");
L.add("b");
```

Today, we'll begin our 4 lecture journey towards building our own list implementations.

- Lists are a useful abstraction.
- Our first example of thinking about tradeoffs of different implementations.

IntList and Linked Data Structures

Lecture 3, CS61B, Fall 2025

Packages

Goals: Building a List

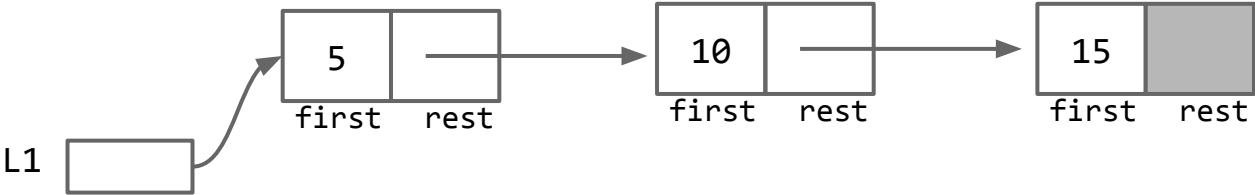
IntList and Linked Data Structures

Let's define an IntList as an object containing two member variables:

- `int first;`
- `IntList rest;`

Let's define an IntList as an object containing two member variables:

- `int first;`
- `IntList rest;`





Write down an example

- What are the example parameters, and return value?
- What would the recursive call(s) return for the example?
- How can you use the result of the recursive call(s)?

This lecture is primarily live-coding. To see the code see:

- <https://github.com/Berkeley-CS61B/lectureCode-sp26>

Non-Destructive:

- A method is non-destructive if it does not modify the object.
 - Example: `public IntList incrementNonDestructive()` returns an incremented version of the current `IntList` without modifying current list.

Destructive

- A method is destructive if it is allowed to modify the object:
 - Example: `public IntList incrementNonDestructive()` returns an incremented version of the current `IntList` without modifying current list.

Coding Demo: Adding to End of IntList

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;
```

```
}
```

Coding Demo: Adding to End of IntList

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    public static void main(String[] args) {  
        IntList L = new IntList();  
        L.first = 5;  
        L.rest = null;  
  
    }  
}
```


Coding Demo: Adding to End of IntList

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    public static void main(String[] args) {  
        IntList L = new IntList();  
        L.first = 5;  
        L.rest = null;  
  
        L.rest = new IntList();  
        L.rest.first = 10;  
  
    }  
}
```

Coding Demo: Adding to End of IntList

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    public static void main(String[] args) {  
        IntList L = new IntList();  
        L.first = 5;  
        L.rest = null;  
  
        L.rest = new IntList();  
        L.rest.first = 10;  
  
        L.rest.rest = new IntList();  
        L.rest.rest.first = 15;  
  
    }  
}
```

[Java Visualizer](#)

Coding Demo: Adding to Start of IntList

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;
```

```
}
```

Coding Demo: Adding to Start of IntList

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    public IntList(int f, IntList r) {  
        first = f;  
        rest = r;  
    }  
}
```

}

Coding Demo: Adding to Start of IntList

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    public IntList(int f, IntList r) {
        first = f;
        rest = r;
    }

    public static void main(String[] args) {
        IntList L = new IntList(15, null);
        L = new IntList(10, L);
        L = new IntList(5, L);
    }
}
```

[Java Visualizer](#)

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    public IntList(int f, IntList r) {
        first = f;
        rest = r;
    }

    public static void main(String[] args) {
        IntList L = new IntList(15, null);
        L = new IntList(10, L);
        L = new IntList(5, L);

        System.out.println(L.size()); // should print out 3
    }
}
```

Coding Demo: IntList size

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    /** Return the size of the list using... recursion! */
    public int size() {

    }

}
```

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the size of the list using... recursion! */  
    public int size() {  
        if (rest == null) {  
  
        }  
  
    }  
  
}
```


IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the size of the list using... recursion! */  
    public int size() {  
        if (rest == null) {  
            return 1;  
        }  
    }  
}
```

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the size of the list using... recursion! */  
    public int size() {  
        if (rest == null) {  
            return 1;  
        }  
        return 1 + this.rest.size();  
    }  
}
```

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    public IntList(int f, IntList r) {
        first = f;
        rest = r;
    }

    public static void main(String[] args) {
        IntList L = new IntList(15, null);
        L = new IntList(10, L);
        L = new IntList(5, L);

        System.out.println(L.iterativeSize()); // should also print out 3
    }
}
```

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    /** Return the size of the list using no recursion! */
    public int iterativeSize() {

    }
}
```

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the size of the list using no recursion! */  
    public int iterativeSize() {  
        IntList p = this;  
  
    }  
  
}
```

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the size of the list using no recursion! */  
    public int iterativeSize() {  
        IntList p = this;  
        int totalSize = 0;  
  
    }  
}
```

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    /** Return the size of the list using no recursion! */
    public int iterativeSize() {
        IntList p = this;
        int totalSize = 0;
        while (p != null) {

        }

    }
}
```

Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    /** Return the size of the list using no recursion! */
    public int iterativeSize() {
        IntList p = this;
        int totalSize = 0;
        while (p != null) {
            totalSize += 1;
        }
    }
}
```


Coding Demo: IntList iterativeSize

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    /** Return the size of the list using no recursion! */
    public int iterativeSize() {
        IntList p = this;
        int totalSize = 0;
        while (p != null) {
            totalSize += 1;
            p = p.rest;
        }
    }
}
```

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the size of the list using no recursion! */  
    public int iterativeSize() {  
        IntList p = this;  
        int totalSize = 0;  
        while (p != null) {  
            totalSize += 1;  
            p = p.rest;  
        }  
        return totalSize;  
    }  
}
```

[Java Visualizer](#)

Challenge

Write a method `int get(int i)` that returns the *i*th item in the list.

- For simplicity, OK to assume the item exists.
- Front item is the 0th item.

Ways to work:

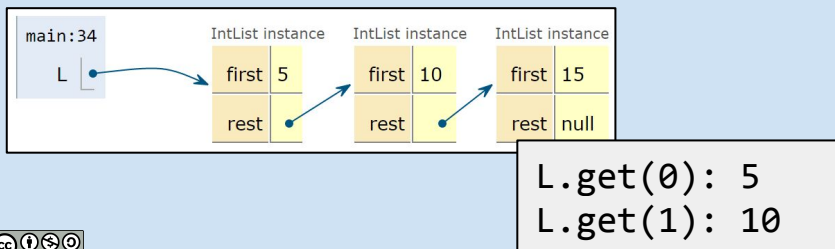
- Paper (best)
- Laptop (see lectureCode repo)
 - exercises/lists1/IntList.java
- In your head (worst)

See the video online for a solution:

https://www.youtube.com/watch?v=qnmxD_21DNk

```
public class IntList {
    public int first;
    public IntList rest;
    public IntList(int f, IntList r) {
        first = f;
        rest = r;
    }

    /** Return the size of this IntList. */
    public int size() {
        if (rest == null) {
            return 1;
        }
        return 1 + this.rest.size();
    }
    ...
}
```



IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    public IntList(int f, IntList r) {
        first = f;
        rest = r;
    }

    public static void main(String[] args) {
        IntList L = new IntList(15, null);
        L = new IntList(10, L);
        L = new IntList(5, L);

        System.out.println(L.get(1)); // should print out 10
    }
}
```

Coding Demo: IntList get

IntList.java

```
public class IntList {
    public int first;
    public IntList rest;

    /** Return the ith item of this IntList. */
    public int get(int i) {

    }

}
```

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the ith item of this IntList. */  
    public int get(int i) {  
        if (i == 0) {  
  
        }  
  
    }  
}
```

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the ith item of this IntList. */  
    public int get(int i) {  
        if (i == 0) {  
            return first;  
        }  
    }  
}
```

IntList.java

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    /** Return the ith item of this IntList. */  
    public int get(int i) {  
        if (i == 0) {  
            return first;  
        }  
        return rest.get(i - 1);  
    }  
}
```

[Java Visualizer](#)

What is your comfort level with recursive data structure code?

- A. Very comfortable.
- B. Comfortable.
- C. Somewhat comfortable.
- D. I have never done this.

The Good:

- Doesn't have a fixed length!
- Implementation was simple

The Bad:

- Takes a lot of effort to go
Through; accessing items is slow

and so is size

```
public class IntList {  
    public int first;  
    public IntList rest;  
  
    public IntList(int f, IntList r) {  
        first = f;  
        rest = r;  
    }  
  
    public static void main(String[] args) {  
        IntList L = new IntList(15, null);  
        L = new IntList(10, L);  
        L = new IntList(5, L);  
    }  
}
```



If you missed attendance QR
earlier, it's

www.yellkey.com/expert

Paper attendance after class if
it's not working for you.